

PROMPT IA 02 - GOLDEN SERVICE

GO : PRODUCT

Construire le premier microservice de reference de bout en bout

Usage	A coller tel quel dans une IA de developpement / coding agent.
Projet	Plateforme e-commerce multi-site OSS-first, Go + Kubernetes + GitOps.
Mode	Execution autonome, incrémentale, testee, sans sur-ingenierie.
Priorite	Code propre, factorise, maintenable, securise et observable.

IMPORTANT: ne pas remplacer les choix techniques valides par des alternatives sans blocage objectif.

1. Mission de l'agent IA

Tu es un senior Go engineer. Implemente le microservice Product comme Golden Service de reference pour toute la plateforme e-commerce. Le code doit etre production-grade, testable, observable, securise et suffisamment simple pour etre duplique par template Backstage. Ne cherche pas a implementer tout le domaine e-commerce : realise un vertical slice complet Product, propre et reellement executable.

Objectif

Le resultat doit servir de modele aux futurs services Catalog, Inventory, Pricing, Tax, Order, Payment, etc. Toute abstraction inutile aujourd'hui deviendra une dette multipliee par le nombre de services : rester minimal et explicite.

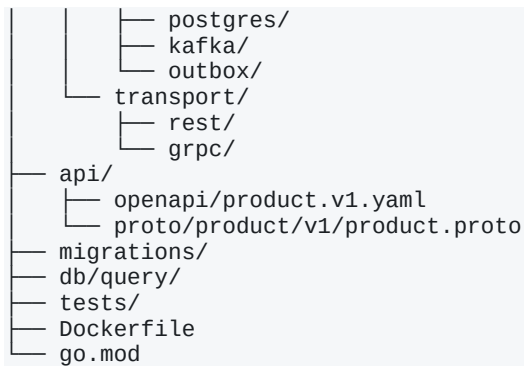
2. Responsabilite metier de Product

Product possede ce qu'est le produit : fiche produit, SKU/variantes, attributs, medias, caracteristiques et prix de reference. Il ne possede ni la presentation/navigation commerciale (Catalog), ni le stock (Inventory), ni le prix effectif/promotions (Pricing).

Product possede	Product ne possede pas
name, description, SKU, variants	categories/navigation/merchandising
attributes/specifications	stock/reservations
media references	promotions/coupons
base/reference price	taxes effectives
lifecycle status	search ranking/facets

3. Architecture interne obligatoire

```
services/product/  
├── cmd/product-api/  
│   └── main.go  
├── internal/  
│   ├── domain/  
│   │   ├── product.go  
│   │   ├── errors.go  
│   │   └── repository.go  
│   ├── application/  
│   │   ├── command/  
│   │   └── query/  
└── infrastructure/
```



REST et gRPC appellent exactement les memes use cases. Le transport ne contient pas de logique metier.

4. Modele Product v1

Commence avec un modele volontairement limite mais extensible.

```
Product
- id: UUIDv7
- sku: string unique
- name: string
- description: string
- status: DRAFT | ACTIVE | ARCHIVED
- base_price_minor: int64
- currency: ISO 4217
- attributes: map[string]string ou structure JSON controlee
- media: []MediaRef
- version: int64
- created_at: timestampz UTC
- updated_at: timestampz UTC

MediaRef
- id: UUIDv7
- type: IMAGE
- url/object_key: string
- position: int
```

Ne stocke pas les fichiers binaires dans PostgreSQL. Product conserve uniquement les references des medias vers le stockage objet.

5. Invariants metier minimaux

- SKU obligatoire, normalise et unique.
- name obligatoire, longueur bornee.
- currency obligatoire et valide.
- base_price_minor ≥ 0 .
- Un produit ARCHIVED ne peut pas etre reactive sans commande explicite prevue par le domaine.
- version incrementee a chaque mutation pour optimistic concurrency.
- Les transitions de status sont explicites, jamais une simple affectation libre depuis HTTP.

6. Use cases a implementer

Use case	REST	gRPC
CreateProduct	POST /api/v1/products	CreateProduct
GetProduct	GET /api/v1/products/{id}	GetProduct
UpdateProduct	PATCH /api/v1/products/{id}	UpdateProduct
ActivateProduct	POST /api/v1/products/{id}/activate	ActivateProduct
ArchiveProduct	POST /api/v1/products/{id}/archive	ArchiveProduct
ListProducts	GET /api/v1/products? limit=&cursor=	ListProducts

ListProducts doit utiliser une pagination cursor-based. Pas d'OFFSET profond.

7. API REST

- OpenAPI 3.1 versionne dans api/openapi/product.v1.yaml.
- JSON strict ; rejeter champs invalides ou payloads trop grands.
- Erreur standard : code, message, trace_id, details optionnels.
- Codes metier stables : PRODUCT_NOT_FOUND, SKU_ALREADY_EXISTS, INVALID_PRODUCT_STATUS, VERSION_CONFLICT.
- ETag/If-Match ou version explicite pour eviter les lost updates sur UpdateProduct.

```
{
  "code": "VERSION_CONFLICT",
  "message": "product version does not match",
  "trace_id": "...",
  "details": {"expected_version": 4}
}
```

8. gRPC

- package product.v1.
- Deadlines obligatoires cote client et respectees cote serveur.
- Mapping coherent des erreurs domaine vers codes gRPC.
- Buf lint + generation + breaking check.
- Ne pas exposer les structs PostgreSQL dans les messages Protobuf.

9. PostgreSQL et sqlc

Product possède sa base logique product_db. Utilise pgx et sqlc. Aucune ORM.

- Migrations SQL versionnées.
- Contraintes DB pour SKU unique, prix ≥ 0 , champs requis et version.
- Index pour SKU, status, created_at/id nécessaires à la pagination cursor.
- Transactions courtes.
- Aucun appel Kafka ou gRPC dans la transaction DB.
- pgxpool borne et configure par environnement.

10. Outbox Pattern et événements Kafka

Toute mutation Product qui doit être diffusée écrit l'événement dans la table outbox dans la même transaction que l'état métier. Un publisher asynchrone publie ensuite dans Kafka.

```
BEGIN
  INSERT/UPDATE product
  INSERT INTO outbox (...)
COMMIT

publisher outbox
↓
Kafka
```

- Events v1 : product.created.v1, product.updated.v1, product.activated.v1, product.archived.v1.
- Envelope standard : event_id, event_type, aggregate_id, aggregate_version, occurred_at, producer, correlation_id, causation_id, trace_id.
- Kafka key = product_id.
- Protobuf comme format.
- Publisher idempotent autant que possible ; consumers responsables de leur idempotence.

11. Projection Search minimale

Le Golden Service doit démontrer la propagation vers Search sans faire de Product un moteur de recherche.

```
Product PostgreSQL
↓
Outbox
↓
Kafka product.*
↓
Search projection consumer
↓
OpenSearch product-read-v1
↓
alias product-read
```

Si le repository Search n'existe pas encore, implémente un consumer/projection minimal dans un module de démonstration ou un test d'intégration, clairement séparé de Product. Ne crée pas de dépendance Product -> OpenSearch directe.

12. Idempotence et concurrence

- CreateProduct supporte Idempotency-Key persistante si appele depuis API publique/backoffice critique.
- Update utilise optimistic concurrency via version.
- Les consumers Kafka dedoublonnent event_id.
- Les retries ne doivent jamais produire deux produits ou doubler un evenement logique.

13. Validation et autorisation

Le frontend n'est jamais autoritaire. Toute validation finale est backend.

- Validation structurelle dans le transport.
- Validation metier dans le domaine/use case.
- Contraintes finales dans PostgreSQL.
- Autorisation basee sur identite/permissions fournies par la couche plateforme ; ne code pas Keycloak dans le domaine.
- Permissions indicatives : product:read, product:write, product:archive.

14. Observabilite

- OpenTelemetry traces REST/gRPC/DB/Kafka.
- slog JSON structure avec trace_id/correlation_id/request_id.
- Metrics RED pour REST/gRPC et metrics outbox/Kafka.
- Aucune PII inutile ou secret dans logs/spans.
- Health endpoints live/ready/startup.

```
metrics additionnelles:  
- product_commands_total{command,result}  
- product_repository_duration_seconds{operation}  
- outbox_pending_total  
- outbox_publish_failures_total  
- outbox_oldest_age_seconds
```

15. Graceful shutdown

```
SIGTERM  
↓  
readiness = false  
↓  
stop new requests  
↓  
finish inflight requests  
↓  
stop outbox publisher / Kafka consumer cleanly  
↓  
close pgx pool  
↓  
exit
```

Aucun ack/commit Kafka ne doit être perdu ou effectué avant que le traitement correspondant soit durablement terminé.

16. Tests obligatoires

Couche	Tests minimaux
Domain	invariants, transitions de status, prix, SKU
Application	use cases avec fakes explicites
Repository	testcontainers PostgreSQL + migrations + sqlc
REST	happy path + validation + erreurs + idempotence
gRPC	status mapping + deadlines
Outbox	atomicité transaction + retry publication
Kafka	Protobuf + envelope + idempotence consumer
Concurrency	optimistic lock/version conflict
Race	go test -race ./...

17. Test E2E vertical slice

1. POST /products
2. vérifier HTTP 201
3. vérifier ligne PostgreSQL
4. vérifier ligne outbox
5. exécuter publisher
6. vérifier événement Kafka product.created.v1
7. consommer événement dans projection Search
8. vérifier document OpenSearch
9. GET /products/{id}
10. vérifier corrélation trace/logs

Le test peut s'appuyer sur testcontainers. Il doit être reproductible localement et dans Tekton.

18. Docker et runtime Kubernetes

- Docker multi-stage.

- Runtime distroless ou equivalent minimal approuve.
- Utilisateur non-root.
- readOnlyRootFilesystem compatible.
- Aucun shell requis dans l'image de production.
- Configuration via variables/env non sensibles ; secrets fournis a runtime par OpenBao.

19. Manifests GitOps de reference

Fournis un exemple declaratif Kustomize/Helm minimal pour Product, sans secret.

- Deployment avec requests/limits.
- HPA.
- PDB.
- topologySpreadConstraints / anti-affinity.
- Service.
- NetworkPolicy Cilium default-deny + flux necessaires.
- Istio Ambient enrollment/policies seulement si necessaire.
- ServiceMonitor/PodMonitor selon stack.
- readiness/liveness/startup probes.

20. Performance et budgets

- REST p95 cible < 300 ms hors dependances externes.
- gRPC p95 cible < 100 ms.
- ListProducts borne et cursor-based.
- Pas de SELECT * sur chemins critiques.
- pg_stat_statements exploitable.
- Toute query critique doit avoir un index justifie et un EXPLAIN teste.

21. Ce qu'il ne faut pas faire

- Pas de generic repository universel.
- Pas de service locator/DI framework complexe.
- Pas de MongoDB.
- Pas de publication Kafka directement depuis le handler REST.
- Pas de logique Search/OpenSearch dans Product.
- Pas de prix promotionnel ni stock dans Product.
- Pas de logique d'authentification Keycloak dans le domaine.
- Pas de secret dans Kubernetes manifests.
- Pas de TODO critique laisse sans test ou ticket explicite.

22. Plan d'execution impose a l'IA

1. Inspecter le monorepo et reutiliser les conventions existantes.
2. Creer modele domaine + tests unitaires.
3. Creer schema PostgreSQL + migrations + sqlc + tests integration.
4. Creer use cases application.
5. Ajouter REST/OpenAPI.
6. Ajouter gRPC/Protobuf.
7. Ajouter Outbox + publisher Kafka + contrats events.
8. Ajouter projection Search minimale/test E2E.
9. Ajouter observabilite, health et graceful shutdown.
10. Ajouter Docker/GitOps/CI puis executer tous les quality gates.

23. Definition of Done

Critere	Attendu
Fonctionnel	CRUD/use cases v1 Product fonctionnent via REST et gRPC.
Persistence	PostgreSQL/sqlc/migrations testes.
Eventing	Outbox -> Kafka product.* valide.
Search	Projection asynchrone demonstrable sans couplage direct.
Tests	Unit + integration + E2E + race passent.
Contracts	OpenAPI/Buf/Protobuf valides.
Security	Runtime non-root, pas de secret, validation backend.
Observability	Logs/traces/metrics exploitables.
Maintainability	Pas de duplication et abstractions minimales.

24. Prompt final a executer

Instruction a l'agent IA

Commence maintenant l'implementation du Golden Service Product dans le monorepo existant. Avance par petites etapes testees. Ne me redemande pas les choix d'architecture deja fournis. Si un detail est ambigu, choisis l'implementation la plus simple qui respecte les invariants et la portabilite. A la fin, execute tous les tests et quality gates, puis fournis : 1) fichiers modifies, 2) commandes executees, 3) resultats de tests, 4) decisions/ADR eventuels, 5) risques restants, 6) prochaine etape.